

trie树

人员

周子一、陈洛冉、谢亚锴、司云心、杨瑾硕、秦显森、于子珈 到课, 李子瀚 线上

上周作业检查

上周作业链接: <https://cppoj.kids123code.com/contest/4523>

王向东老师周日十点半C++逆元练习							
刷新							
#	用户名	姓名	编程分	时间	A	B	C
1	yuzijia1	于子珈	300	2492	100	100	100
2	lizihan	李子瀚	300	4421	100	100	100
3	yangjinshuo	杨瑾硕	200	232	100	100	
4	zhouziyi	周子一	200	234	100	100	
5	qinxiansen	秦显森	200	237	100	100	
6	chenluoran	陈洛冉	200	244	100	100	
7	xieyakai	谢亚锴	200	518	100	100	

本周作业

<https://cppoj.kids123code.com/contest/4675> (课上讲了 A ~ D 题, 课后作业是 C D 题必做)

课堂表现

今天课上讲了 trie 树这个知识点, 知识点不难, 同学们课上都掌握的比较好, 但是这个知识点很容易遗忘, 同学们课下要多复习复习这个算法。

课堂内容

【模板】字典树

trie 树模板题

tr[i][j]: 编号为 i 的点的 j 号孩子的编号是多少

```
#include <bits/stdc++.h>

using namespace std;

const int N = 3e6 + 5, M = 62;
int tr[N][M], cnt[N], idx = 0;

int get_int(char x) {
    if (islower(x)) return x - 'a';
    if (isupper(x)) return x - 'A' + 26;
```

```
    return x-'0'+52;
}

void tr_insert(string s) {
    int p = 0;
    for (char i : s) {
        int u = get_int(i);
        if (!tr[p][u]) tr[p][u] = ++idx;
        p = tr[p][u];
        ++cnt[p];
    }
}

int tr_query(string s) {
    int p = 0;
    for (char i : s) {
        int u = get_int(i);
        if (!tr[p][u]) return 0;
        p = tr[p][u];
    }
    return cnt[p];
}

void solve() {
    int n, m; cin >> n >> m;
    while (n -- ) {
        string s; cin >> s; tr_insert(s);
    }

    while (m -- ) {
        string s; cin >> s;
        cout << tr_query(s) << "\n";
    }

    for (int i = 0; i <= idx; ++i) {
        cnt[i] = 0;
        for (int j = 0; j < M; ++j) tr[i][j] = 0;
    }
    idx = 0;
}

int main()
{
    int T; cin >> T;
    while (T -- ) solve();
    return 0;
}
```

[NOI2000] 单词查找树

```
#include <bits/stdc++.h>

using namespace std;

const int N = 3e6 + 5, M = 62;
int tr[N][M], cnt[N], idx = 0;

int get_int(char x) {
    if (islower(x)) return x - 'a';
    if (isupper(x)) return x - 'A' + 26;
    return x - '0' + 52;
}

void tr_insert(string s) {
    int p = 0;
    for (char i : s) {
        int u = get_int(i);
        if (!tr[p][u]) tr[p][u] = ++idx;
        p = tr[p][u];
        ++cnt[p];
    }
}

int tr_query(string s) {
    int p = 0;
    for (char i : s) {
        int u = get_int(i);
        if (!tr[p][u]) return 0;
        p = tr[p][u];
    }
    return cnt[p];
}

int main()
{
    string s;
    while (cin >> s) tr_insert(s);
    cout << idx + 1 << endl;
    return 0;
}
```

最大异或对 The XOR Largest Pair

从 1~i 遍历, trie 树中存前 i-1 个点的二进制

对于第 i 个点, 去前面找跟 a[i] 的二进制尽量相反的二进制

```
#include <bits/stdc++.h>

using namespace std;
```

```

const int N = 100000 + 5, M = 32;
int tr[N*M][2], idx = 0;

void tr_insert(int x) {
    int p = 0;
    for (int i = 31; i >= 0; --i) {
        int u = (x>>i)&1;
        if (!tr[p][u]) tr[p][u] = ++idx;
        p = tr[p][u];
    }
}

int tr_query(int x) {
    int p = 0, res = 0;
    for (int i = 31; i >= 0; --i) {
        int u = (x>>i)&1;
        if (tr[p][u^1]) p = tr[p][u^1], res += ((u^1)<<i);
        else p = tr[p][u], res += (u<<i);
    }
    return res;
}

int main()
{
    int n; cin >> n;
    int res = 0;
    for (int i = 1; i <= n; ++i) {
        int x; cin >> x; tr_insert(x);
        int t = tr_query(x);
        res = max(res, x^t);
    }
    cout << res << endl;
    return 0;
}

```

最长异或路径

先 dfs 求出根到每个点的异或值, 然后问题就转化为上一个问题了

点 v_1 到 v_2 之间的路径异或值, 就是 根到 v_1 的异或值 $\wedge$ 根到 v_2 的异或值

```

#include <bits/stdc++.h>

using namespace std;

const int maxn = 100000 + 5;
struct node {
    int to, value;
};
vector<node> vec[maxn];

```

```
int tr[maxn*32][2], idx = 0;
void tr_insert(int x) {
    int p = 0;
    for (int i = 31; i >= 0; --i) {
        int u = (x>>i)&1;
        if (!tr[p][u]) tr[p][u] = ++idx;
        p = tr[p][u];
    }
}
int tr_query(int x) {
    int p = 0, res = 0;
    for (int i = 31; i >= 0; --i) {
        int u = (x>>i)&1;
        if (tr[p][u^1]) p = tr[p][u^1], res += ((u^1)<<i);
        else p = tr[p][u], res += (u<<i);
    }
    return res;
}

int f[maxn], res = 0;
void dfs(int u, int fa, int val) {
    f[u] = val;

    tr_insert(val);
    int val2 = tr_query(val);
    res = max(res, val^val2);

    for (node it : vec[u]) {
        if (it.to != fa) dfs(it.to, u, val^it.value);
    }
}

int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n-1; ++i) {
        int u, v, w; cin >> u >> v >> w;
        vec[u].push_back({v, w}), vec[v].push_back({u, w});
    }

    dfs(1, -1, 0);

    cout << res << endl;
    return 0;
}
```